



南方科技大学

LAB 9: LoRA & Fine-Tuning

COE 201 - AI Large Models, Spring 2026

2026-04-27

College of Engineering, SUSTech

Office Hour: Thursday, 14:30-15:30 @ 3rd Research Building, Room 610

Outline

- Supervised Fine-Tuning (SFT) 1
- LoRA: Low-Rank Adaptation 8
- The PEFT Library 14
- Lab 9 Exercises 20

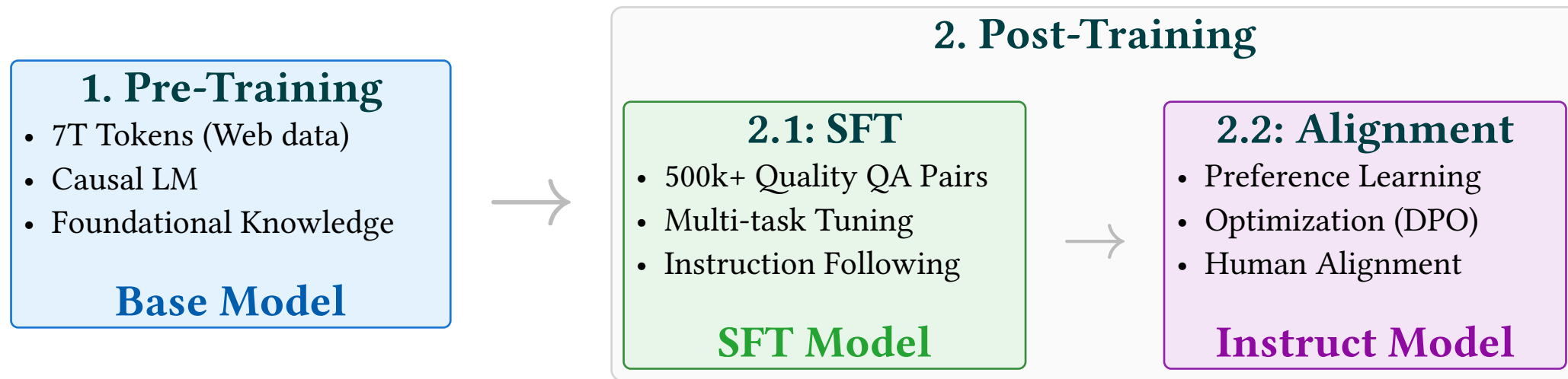
Supervised Fine-Tuning (SFT)

The Modern LLM Pipeline



南方科技大学

Based on Qwen2 Tech Report: <https://arxiv.org/pdf/2407.10671>



- **Base models** (e.g. Qwen) are released by vendors; pre-training costs millions of USD.
- **This Lab:** We focus on **Post-Training** (SFT & Alignment) using LoRA (Low-cost fine-tuning).

Supervised Fine-Tuning (SFT): The Data



SFT trains the model on curated (prompt, response) pairs to shift its behavior from predicting internet text to following user instructions.

Messages List (JSON)

```
[
  {
    "role": "system",
    "content": "You are a helpful assistant."
  },
  {
    "role": "user",
    "content": "Classify the sentiment: Great movie!"
  },
  {
    "role": "assistant",
    "content": "Positive"
  }
]
```

JSON



Chat Template Formatted

```
<|im_start|>system
You are a helpful assistant.<|im_end|>
<|im_start|>user
Classify the sentiment: Great movie!<|im_end|>
<|im_start|>assistant
Positive<|im_end|>
```

Text

Note: Modern workflows use `tokenizer.apply_chat_template()` to automatically convert messages into the model's specific formatting structure (like ChatML above).

The Alpaca Dataset: Synthetic SFT Data



Stanford's Alpaca project (2023) introduced an instruction-following dataset.

Self-Instruct Dataset

- 52k (instruction, response) pairs
 - Generated by GPT-3.5
 - Cost less than \$600 to generate
-
- **instruction**: describes the task the model should perform.
 - **input**: context or input for the task.
 - **output**: the answer to the instruction.
 - **text**: formatted prompt (can be replaced by chat template).

Example:

```
1 {
2   "instruction": "Create a classification task by
3   clustering the given list of items.",
4   "input": "Apples, oranges, bananas, strawberries,
5   pineapples",
6   "output": "Class 1: Apples, Oranges\nClass 2: Bananas,
7   Strawberries\nClass 3: Pineapples",
8   "text": "Below is an instruction that describes a task,
9   paired with an input that provides further context.
10  Write a response that appropriately completes the
11  request. .... [omitted for brevity] ....",
12 }
```

Loss: Cross Entropy on Response tokens

$$\text{CrossEntropy} = - \sum_{\{t\}} \log P(y_t \mid y_{\{<t\}}, x)$$

Computed **only** on Response (Prompt is masked).

```
1 # Training loop (simplified)
2 for batch in dataloader:
3     # Teacher forcing: predict each token
4     # given all previous tokens
5     logits = model(batch["input_ids"])
6     # loss on (prediction, shifted target)
7     loss = cross_entropy(logits[:, :-1, :], labels[:, 1:])
8     loss.backward()
9     optimizer.step()
```

Python

Teacher Forcing Training

- Uses **ground truth** past tokens, not model's own predictions.
- Enable stable, parallel computation.

Label Shift & Prompt Masking

Pre-training predicts **all** tokens. SFT predicts **only** the response.

Input: [<user> Hi <asst> Hello !]

Label: [<pad> <pad> <pad> Hello ! <eos>]

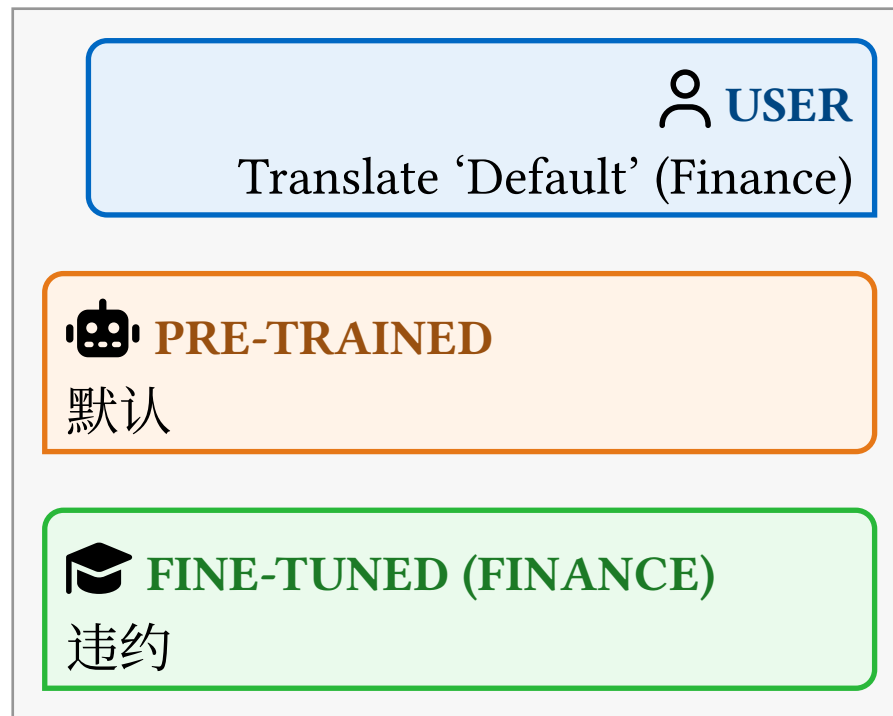
Downstream Fine-Tuning



Pre-trained models are **general-purpose** — they know a bit about everything, but may lack expertise in your specific domain:

- **Medical / Legal**: specialized terminology the model rarely saw.
- **Internal Data**: company docs, proprietary APIs, private knowledge.
- **etc.**

Solution: Fine-tune the model on our own domain-specific data using SFT!



An example of general-purpose model vs. domain-specific fine-tuned model.

The Full Parameter Problem



南方科技大学

Why don't we just update all weights for every task?

Memory Wall

Full gradients and optimizer states must be stored for **every** parameter.

- **Model (7B):** 14 GB (FP16)
- **Optimizer (Adam):** 28-84 GB
- **Total: 40-100 GB VRAM**

→ Needs high-end A100/H100.

Storage Wall

Each fine-tuned model is a **complete copy** of the original.

- 1 task: 14 GB
- 10 tasks: 140 GB
- 100 tasks: 1.4 TB

→ Hard to deploy many specialized models.

Catastrophic Forgetting

Updating all weights shifts the model's manifold too far.

- Loses general knowledge.
- Overfits to small SFT datasets.

Can we fine-tune with $< 1\%$ of the parameters?

LoRA: Low-Rank Adaptation

Freeze the main model, train a tiny bypass.

For any pre-trained **linear layer** $W \in \mathbb{R}^{d \times k}$, full fine-tuning learns an update ΔW :

$$h = W'x = (W + \Delta W)x, \quad \text{where } W \in \mathbb{R}^{d \times k} \text{ and } \Delta W \in \mathbb{R}^{d \times k}$$

Key Insight of LoRA: The update ΔW has a **low intrinsic rank** — it doesn't need all $d \times k$ degrees of freedom. We can approximate it with two much smaller matrices:

$$\Delta W \approx BA, \quad \text{where } A \in \mathbb{R}^{r \times k}, B \in \mathbb{R}^{d \times r}, \quad r \ll \min(d, k)$$

So LoRA **freezes** W and **injects additional trainable** A, B as a bypass:

$$h = \underbrace{W}_{\text{frozen}} x + \underbrace{BA}_{\text{trainable}} x$$

LoRA: Efficiency vs. Performance



南方科技大学

LoRA v.s. Full Fine-Tuning

With $\sim 0.4\%$ trainable parameters, LoRA SFT can achieve a similar performance.

Full Fine-Tuning

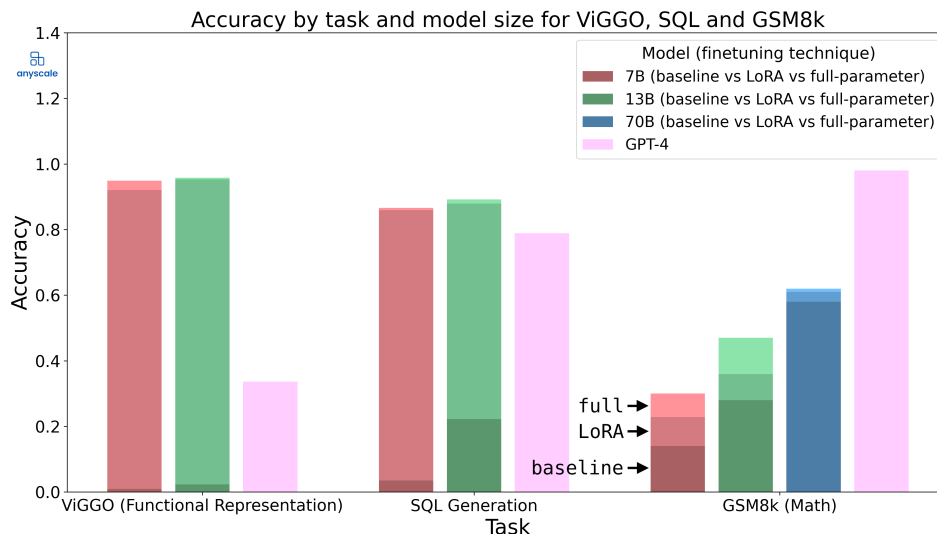
Update all $d \times k$ parameters.

e.g., $4096 \times 4096 = 16.8M$ params per layer

LoRA ($r = 8$)

Update only $(d + k) \times r$ parameters.

e.g., $(4096 + 4096) \times 8 = 65.5K$ params $\rightarrow$
256 times fewer



Model performance on domain-specific tasks
(Baseline v.s. Full-parameter SFT v.s. LoRA)

[Source](#)

The initial state must not change the model's output:

```
1 class LoRALinear(nn.Module):  
2     def __init__(self, linear, r=8, alpha=8):  
3         super().__init__()  
4         self.linear = linear  
5         self.scaling = alpha / r  
6  
7         # A: random init (Kaiming)  
8         self.lora_A = nn.Parameter(torch.empty(r, in_features))  
9         nn.init.kaiming_uniform_(self.lora_A)  
10  
11        # B: zero init => delta_W = BA = 0  
12        self.lora_B = nn.Parameter(torch.zeros(out_features, r))  
13  
14        # Freeze original weights  
15        linear.weight.requires_grad_(False)
```

Python

Why $B = 0$?

At initialization:

$$\Delta W = BA = 0 \cdot A = 0$$

The model starts behaving **exactly** like the original. Training then gradually learns the correction ΔW .

The Scaling Factor



The LoRA output is scaled by α/r :

$$y = Wx + (\alpha/r)BAx$$

α (lora_alpha)

A fixed hyperparameter controlling the overall magnitude of the LoRA update.

Common values: 16, 32.

r (rank)

Controls the expressiveness of the adaptation.
Higher r = more capacity but more parameters.

Common values: 4, 8, 16, 64.

Practical tip: Setting $\alpha = 2 \times r$ is a common starting point. This makes the scaling factor = 2, giving the LoRA path moderate influence.

Recap: Attention & LoRA Targets



$$\text{Attention Mechanism: } \text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where $Q = XW_q$, $K = XW_k$, $V = XW_v$, and the final output uses W_o .

LoRA can be used on any linear layer. Where does LoRA attach?

Standard LoRA Targets

- W_q (q_proj): Query projection
- W_v (v_proj): Value projection

Adapting just Q and V is the **industry standard** balance of efficiency and performance.

Possible Extended Targets

- W_k (k_proj): Key projection
- W_o (o_proj): Output projection
- FFN layers (SwiGLU/MLP, ...)

All linear layers, uses more parameters.

The PEFT Library

Hugging Face's standard toolkit for parameter-efficient fine-tuning.

Inspecting the Model Architecture



Before applying LoRA, we must identify the names of the **target layers** in the model:

```
1 from transformers import AutoModelForCausalLM
2 model = AutoModelForCausalLM.from_pretrained("Qwen/Qwen2.5-0.5B")
3 print(model) # to get the model architecture, and the names of the linear layers
4 """
5 Qwen2ForCausalLM(
6   (model): Qwen2Model(
7     ...
8     (q_proj): Linear(...)
9     (v_proj): Linear(...)
10    ...
11  """
```

Python

Goal: Find the names of the `Linear` modules you want to adapt (e.g., `q_proj`, `v_proj`). These names vary between model architectures (Llama, Qwen, etc.).

PEFT: Injecting Adapters



Hugging Face `peft` provides an interface to wrap your model with LoRA adapters:

```
1  from peft import LoraConfig, get_peft_model
2
3  # 1. Define configuration
4  lora_config = LoraConfig(
5      r=8,
6      target_modules=["q_proj", "v_proj"], # Match the names from print(model)
7  )
8  # 2. Wrap the model
9  model = get_peft_model(model, lora_config)
10 model.print_trainable_parameters()
11 # trainable params: 1.04M || all params: 614M || 0.17%
```

Python

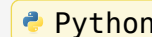
Key Insight: `peft` automatically freezes the base model and attaches the bypass matrices to the modules specified in `target_modules`.

Advanced: Targeting Specific Layers



Sometimes we only want to adapt the **high-level semantic layers** (e.g., the last 4 layers) instead of the whole model. This is controlled via `layers_to_transform`.

```
1  from peft import LoraConfig, get_peft_model
2
3  # Qwen2.5-0.5B has 24 layers (indexed 0 to 23)
4  config = LoraConfig(
5      r=16,
6      target_modules=["q_proj", "v_proj"],
7      # Apply LoRA ONLY to the last 4 layers (20, 21, 22, 23)
8      layers_to_transform=list(range(20, 24)),
9  )
10 model = get_peft_model(base_model, config)
```



Why?: The initial layers often capture general language features, while the final layers are more task-specific. Targeting only the end can save even more memory!

The Training Loop with Hugging Face Trainer



南方科技大学

```
1 from transformers import Trainer, TrainingArguments, DataCollatorForLanguageModeling
```

Python

```
2 # 1. Training Configuration
```

Python

```
3 training_args = TrainingArguments(  
4     output_dir="./lora_output",  
5     num_train_epochs=1,  
6     per_device_train_batch_size=4,  
7     gradient_accumulation_steps=4,  
8     learning_rate=2e-4,  
9     fp16=True,  
10 )
```

```
11 # 2. Data Collator & Trainer
```

Python

```
12 data_collator = DataCollatorForLanguageModeling(  
13     tokenizer=tokenizer, mlm=False  
14 )  
15 trainer = Trainer(  
16     model=model,  
17     args=training_args,  
18     train_dataset=tokenized_dataset,  
19     data_collator=data_collator,  
20 )  
21 trainer.train()
```

LR Tip: LoRA updates are small, so use a larger LR (2×10^{-4}) compared to full fine-tuning (2×10^{-5}).

LoRA adapters are tiny — only the trained A and B matrices are saved:

```
1  # Save only the adapter (tiny!)
2  model.save_pretrained("./my_lora_adapter")
3  # Disk usage: ~4 MB (vs ~1 GB for full model)
4
5  # Load base model + adapter
6  base_model = AutoModelForCausalLM.from_pretrained(
7      "Qwen/Qwen2.5-0.5B"
8  )
9  model = PeftModel.from_pretrained(
10     base_model,
11     "./my_lora_adapter"
12 )
```

Python

Benefits of Small Adapters

- **Storage:** 4 MB vs 1 GB per model
- **Sharing:** Upload adapters to HF Hub
- **Composing:** Swap adapters for different tasks on the same base model
- **Merging:** Combine adapter into base weight for zero-overhead inference

Lab 9 Exercises

Implement LoRA from scratch and fine-tune a real model.

Task 0: Environment Setup (AutoDL)



南方科技大学

To save time and disk space, we have provided shared models and datasets on the AutoDL Beijing region storage.

1. Server Selection

- 地区: 西北 B 区 or 北京 B 区, 镜像: PyTorch 2.8
- `pip install transformers peft datasets accelerate matplotlib numpy`
- GPU: **RTX 4090, RTX 5090**, or **vGPU 32G**

2. Link Shared Assets

Run the following command in your lab directory to link the pre-downloaded models and data: `ln -s /root/autodl-fs/C0E201_lab/lab9/* .`

Task 0: Environment Setup (AutoDL)



南方科技大学

Task 1: LoRA Layer from Scratch (`lab9_lora_layer.py`) [40 points]

- Implement the `LoRALinear`: freeze original weights, and create A & B matrices with correct initialization.
- Implement the forward pass: $y = Wx + (\alpha/r)BAx$.
- Use `inject_lora` to wrap a simple model and verify:
 - Original weights are frozen.
 - LoRA starts as identity ($B = 0 \rightarrow \Delta W = 0$).
 - Trainable parameter count is correct.

Task 0: Environment Setup (AutoDL)



南方科技大学

Task 2: SFT Fine-Tuning with LoRA (`lab9_sft_finetune.py`) [30 points]

- Load the Alpaca dataset and tokenize it into the instruction format.
- Use `peft` to apply LoRA to Qwen2.5-0.5B.
- Train the model (3 epochs by default) using the Hugging Face `Trainer`.
- Generate sample output before and after training.

Try the Chatbot: Once Task 2 is done, run `lab9_chatbot.py` to chat with your fine-tuned model and compare it with the base model!

Task 0: Environment Setup (AutoDL)



南方科技大学

Task 3: Manual LoRA Weight Merging (`lab9_merge_lora.py`) [30 points]

- Load the base model and the LoRA adapter safetensors saved in Task 2.
- Manually compute $W_{\text{merged}} = W_{\text{base}} + (BA)(\alpha/r)$ and update the base weights.
- Prove that your manually merged model achieves the fine-tuned output with zero inference overhead.

Task 0: Environment Setup (AutoDL)



南方科技大学

Task 4: Chatbot Performance Tuning [Bonus]

This may take much more time to train, so only if you have finished task 1-3.

- **Goal:** Train a chatbot that follows complex instructions well.
- You are supposed to understand the full skeleton code in `lab9-stf-finetune.py` and try to optimize its settings (not only code snippet you finished in task 2).
- **Experiment with:**
 - Increasing dataset size (e.g., `train[:10000]`)
 - Increasing epochs to train.
 - Tuning LoRA r and `target_modules`.
 - etc...

Happy Coding!